Department of Information Science and Engineering

Faculty of Engineering & Technology

Jain Global Campus, Kanakapura Taluk - 562112

Ramanagara District, Karnataka, India

Artificial Intelligence

(23ISEDE11)

Experiential Learning report

on

"Designing an Intelligent Agent to Navigate a Wumpus World using Heuristic Search and Logical Reasoning"

Submitted by :

Name : Jeffrey Thomas

USN: 23BTRIS016

Semester : 5th sem

Submitted to :

Mrs. V. Nirupa,

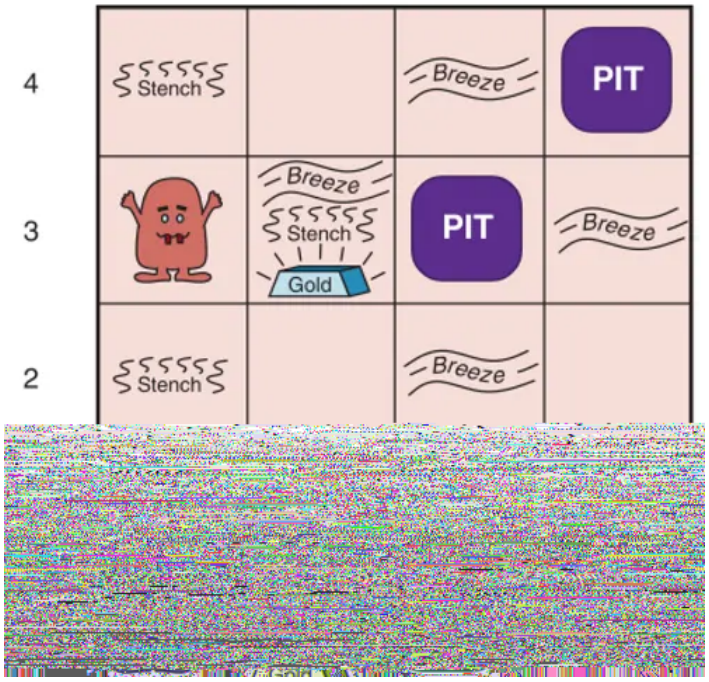
Dept.of ISE,

Faculty of Engineering & Technology

Jain Global Campus, Kanakapura Taluk - 562112

Ramanagara District, Karnataka, India

1. Introduction : What is a Wumpus World?



Wumpus World is a 4x4 grid consisting of **16 rooms**. Agent starts at Room[1,1] facing right and its goal is to retrieve treasure while avoiding hazards such as pits and the Wumpus. Agent must navigate through grid using its limited sensory input to make decisions that will keep it safe and allow it to successfully collect the treasure and exit the cave.

Key Elements:

- **Pits**: If the agent steps into a pit it falls and dies. A **breeze** in adjacent rooms suggests nearby pits.
- **Wumpus**: A creature that kills agent if it enters its room. Rooms next to the Wumpus have a **stench**. Agent can use an **arrow** to kill the Wumpus.
- **Treasure**: Agent's main objective is to **collect the treasure** (gold) which is located in one room.
- **Breeze**: Indicates a pit is nearby.
- **Stench**: Indicates the Wumpus is nearby.

2.PEAS Description

PEAS stands for **Performance Measures**, **Environment**, **Actuators** and **Sensors** which describe agent's capabilities and environment.

1. Performance measures: Rewards or Punishments

- Agent gets gold and return back safe = *+1000 points*
- Agent dies (pit or Wumpus)= *-1000 points*
- Each move of the agent = *-1 point*
- Agent uses the arrow = *-10 points*

2. Environment: A setting where everything will take place.

- A cave with *16(4x4)* rooms.
- Rooms adjacent (not diagonally) to the Wumpus are stinking.
- Rooms adjacent (not diagonally) to the pit are breezy.
- Room with gold glitters.
- Agent's initial position - Room[1, 1] and facing right side.
- Location of Wumpus, gold and 3 pits can be anywhere except in *Room[1, 1]*.

3. Actuators: Devices that allow agent to perform following actions in the environment.

- Move forward: Move to next room.
- Turn right/left: Rotate agent 90 degrees.
- Shoot: Kill Wumpus with arrow.
- Grab: Take treasure.
- Release: Drop treasure

4. Sensors: Devices help the agent in sensing following from the environment.

- Breeze: Detected near a pit.
- Stench: Detected near the Wumpus.
- Glitter: Detected when treasure is in the room.
- Scream: Triggered when Wumpus is killed.
- Bump: Occurs when hitting a wall.

2.2 How the Agent Operates with PEAS

1. **Perception**: Agent uses sensory inputs (breeze, stench, glitter) to detect its surroundings and understand the environment.
2. **Inference**: Agent applies logical reasoning to find location of hazards. For example if it detects a **breeze**, it warns that a pit is nearby or if there's a **stench** it suspects the Wumpus is in an adjacent room.
3. **Planning**: Based on its deductions agent plans its next move avoiding risky areas like rooms with suspected pits or the Wumpus.
4. **Action**: Agent performs planned action such as moving to a new room, shooting arrow at the Wumpus or taking the treasure.

This process repeats till the agent finds the cave using its sensory inputs, reasoning and planning to achieve its goal safely.

By using **PEAS** framework agent's interactions with its environment are clearly defined, providing a structured approach to modeling intelligent behavior.

3. Key Properties of Wumpus World

Wumpus World has distinct characteristics that define how the agent interacts with the environment and makes decisions. These properties influence the agent's strategy, perception, and movement.

- **Partially Observable:** The agent does not have full visibility of the grid; instead, it perceives clues (breeze, stench, glitter) from adjacent cells. It must infer hidden dangers using logical reasoning.

- **Deterministic:** The environment follows fixed rules, meaning that the agent's actions have predictable outcomes. If the agent moves forward, it will land in the expected cell unless an obstacle prevents it.

- **Static:** The Wumpus and pits remain stationary throughout the game, making it easier for the agent to map hazards and plan movements. Unlike dynamic environments, the agent does not need to adapt to moving threats.

- **Discrete:** The environment consists of a 4×4 grid with distinct cells, where the agent moves step by step. Unlike continuous environments, actions are limited to specific choices within defined spaces.

4. Representing Knowledge Using Propositional Logic

We use logic rules to represent dangers.

Example Rules :

- If there is a **breeze** at a tile, then at least one adjacent tile has a **pit :**

$$B(x,y) \rightarrow P(x+1,y) \lor P(x-1,y) \lor P(x,y+1) \lor P(x,y-1)$$

- If there is **no breeze**, then all adjacent cells have **no pits :**

$$\neg B(x,y) \rightarrow \neg P(x+1,y) \land \neg P(x-1,y) \land \neg P(x,y+1) \land \neg P(x,y-1)$$

- Similar rules exist for the **Wumpus** using **stench.**

5. Heuristic Search

The agent uses two major algorithms:

1. Greedy Best-First Search

- Always picks the tile that *looks closest* to the goal (gold).
- Uses only heuristic value **h(n)** (estimated distance).
- Fast but not always optimal (may get stuck in wrong paths).

2. A Search (A-star)*

- Considers two things:
  - **g(n)** → cost already traveled
  - **h(n)** → estimated distance to goal
- Uses formula:
  f(n) = g(n) + h(n)
- Always gives an optimal and safe path if heuristic is good.

The agent approximates how far the gold is using:

```
Manhattan Distance = |x1 - x2| + |y1 - y2|
```

This tells the agent:

- which tile is **closest**
- which next move will bring it **nearer to the goal**

Example:

If gold is at (4,4) and agent is at (1,1):
Distance = |4–1| + |4–1| = 6

6. Logical Reasoning

Logical reasoning allows the agent to **think**, **infer**, and **make decisions** based on the sensations (percepts) it receives inside the Wumpus World.
The agent doesn't see the entire map. It only gets clues like **breeze**, **stench**, and **glitter**. Using these clues, it forms *logical conclusions* about which tiles are **safe** and which tiles may contain **pits** or the **Wumpus**.

What the agent does with logical reasoning:

1. **Detects safe cells**
   If a tile has *no breeze* and *no stench*, the agent concludes all nearby tiles are safe.

2. **Identifies possible dangers**
   If a breeze is felt, the agent concludes that *one of the nearby tiles* has a pit.
   If a stench is felt, the agent concludes that *one of the nearby tiles* has the Wumpus.

3. **Updates its Knowledge Base (KB)**
   Each time the agent enters a tile, it stores the new percepts in its memory.
   Example:
   If it visited (1,1) and no breeze was detected, it stores:
   ¬Breeze(1,1) and **no pit in neighbors**.

4. **Infers hidden facts**
   Using logical rules, the agent can figure out facts that were not directly perceived.
   Example:
   If (1,2) has a breeze AND (1,3) has no breeze → pit must be in (2,2).

7. Logical Reasoning Process

7.1 Forward chaining(facts —> goals)

Idea: start from facts (percepts) and repeatedly apply rules to *add* new facts to the KB until nothing new appears.

How it works :

1. Agent **perceives** at current cell: e.g., sees B[1,1] = False, S[1,1] = False.
2. Insert these **percepts into KB:** ¬B[1,1], ¬S[1,1].
3. **Apply rules**: ¬B[1,1] → ¬P[1,2] ∧ ¬P[2,1] ⇒ add ¬P[1,2], ¬P[2,1].
4. From ¬S[1,1] → ¬W[1,2] ∧ ¬W[2,1] ⇒ add ¬W[1,2], ¬W[2,1].
5. **Combine:** for neighbor (1,2), we now have ¬P[1,2] ∧ ¬W[1,2] ⇒ add Safe[1,2].
6. **Agent now knows (1,2) is safe** — forward chaining produced that.

7.2 Backward chaining (goal → needed facts)

Idea: start from a *query* (e.g., "Is cell (2,2) safe?") and try to prove it by looking for rules that support it, recursively proving their prerequisites.

How it works:

To prove Safe[2,2]:

1. Safe[2,2] can be proved if we can prove ¬P[2,2] and ¬W[2,2].
2. To prove ¬P[2,2], look for a rule that gives ¬P[2,2], for example ¬B[2,1] → ¬P[2,2]. So we need to prove ¬B[2,1].
3. ¬B[2,1] is known if we have percept B[2,1]=False (agent has already visited (2,1)). If it's known, we succeed. If not, we may try other rules or fail.

7.3 Resolution (proof by contradiction)

Idea: Use propositional logic in Conjunctive Normal Form (CNF).

To show A is true, try to show KB ∧ ¬A leads to contradiction (derive empty clause) using resolution. Resolution proves *unsatisfiability* of KB ∧ ¬A.

Steps:

1. Convert KB and ¬A to CNF clauses.
2. Repeatedly apply *resolution rule*: from Clause1: (L ∨ ...) and Clause2: (¬L ∨ ...) produce resolvent with L removed.
3. If you derive empty clause (), contradiction found → A is entailed by KB.

Tiny resolution example:

Suppose KB contains:

- ¬B[1,1] (no breeze at 1,1)
- Rule: ¬B[1,1] → ¬P[1,2] which in CNF is B[1,1] ∨ ¬P[1,2] (because A→B is ¬A ∨ B; here ¬B → ¬P becomes B ∨ ¬P)
  We want to prove ¬P[1,2]:
- Add P[1,2] as assumption to KB (i.e., add P[1,2] or its CNF clause).
- Now resolve B[1,1] ∨ ¬P[1,2] with ¬B[1,1] (which as clause is ¬B[1,1]) — resolve on B[1,1] to get ¬P[1,2].
- Resolving ¬P[1,2] with P[1,2] yields empty clause → contradiction. So ¬P[1,2] must be true.

This shows resolution can *prove* ¬P[1,2] by contradiction.